Time complexity, Searching, Sorting

1. Calculate the time complexity of the following code snippet:

```
public static void func(int[] arr) {
    int n = arr.length;
    int k = 0;

    for (int i = 0; i < n; i++) {
        k += arr[i];
    }

    for (int i = 1; i < n; i++) {
        for (int j = 1; j <= arr[i]; j++) {
            System.out.print(i);
        }
    }
}
```

2.

Calculate the time complexity of the following code snippet:

```
int b = 0;

for (int i = n; i > 0; i -= 2) {
    for (int j = n; j > 0; j--) {
        b += i * j;
    }
}

for (int k = 1; k < n; k *= 4) {
    for (int m = 0; m < n; m += 3) {
        if (m == 10) {
            break;
        }

        b += k + m;
    }
}

return b;
```

3.

You are working with a weather monitoring station that records the temperature every hour for 12 hours during the night shift. During the night, the temperature gradually drops to reach the coldest hour, and then begins to rise again. Your task is to find the minimum temperature of the night, which corresponds to the coldest hour.

The station tracks temperature every hour over a 12-hour period. A sample pattern of the temperature reading per hour is given here:

Hour	1	2	3	4	5	6	7	8	9	10	11	12
Temperature (°C)	28	24	20	16	12	10	9	11	15	19	23	27

In this example, the temperature decreases from hour 1 to hour 7 (coldest at 9°C). Then it increases from hour 8 to hour 12.

a. **Propose** a linear search approach to find the minimum temperature. Present the solution with pseudocode/programmable code/step-by-step logical instructions.

b. Now **propose** a modification to binary search to solve the same problem. Present the solution with pseudocode/programmable code/step-by-step logical instructions.

.

4 .

Consider a farm with **n** cows of two colors, say, white cows and black cows. The cows are arranged in their barn in such a way that all white cows are kept first, followed by all black cows. Each of the cows is assigned a serial number. **You are to find the serial number of the first black cow.**

a. **Describe** how to convert the given information into an array of integers.

b. Given the array of integers from Q(2a), **present** a method with a time complexity **less than O(n)** to solve the mentioned problem in the paragraph. Use pseudocode/ executable code/ step-by-step logical instructions to show your method.

5.

You are participating in a competitive multiplayer strategy game where the objective is to defeat your opponent within a fixed time constraint. Throughout the game, various perks are provided at each level in the form of health boxes. At **Level 1**, you are presented with a list of health boxes, each represented by an integer. Positive values indicate a health boost to your own character, while negative values correspond to a reduction in your opponent's health. At this level, you are allowed to select exactly one box, and you want to maximize the health damage to your opponent. This list of health points is given below:

[2, 3, -4, -3, 5, -1, 2, -2]

Here, each element corresponds to a distinct health box.

For the above scenario, can you find the box that will cause the maximum possible reduction in enemy health by applying a **divide and conquer** approach? **Explain** your idea in brief, *no need to write full code*.

a. At **Level 2**, you are offered the same collection of health boxes. However, this time you are allowed to choose **one or a block of consecutive boxes without gaps**. Now you want to increase your own health by collecting points that yield the highest points gain. You came up with an algorithm with **O(N*logN)** time complexity to compute this optimal health gain. Where 'N' is the number of health boxes.

b. **Show** a simulation of the algorithm with proper steps.

6. A. Consider the following circular array where the values are in ascending order starting from index 3 and the size of the array is 8. You were asked to write an algorithm to search a value in the array.

Index	0	1	2	3	4	5	6	7
Value	10	11	13	1	3	5	6	9

Present your solution idea as a code/pseudocode/flowchart/step by step instructions/ logical explanations in short.

a. Write the time complexity of your presented solution.
b. Show simulation of the algorithm idea you presented in (a) on the given circular array.

7.

You are given an array B consisting of N integers. Some of the numbers are even, and some are odd. For example B : {7, 2, 9, 4, 11, 6}.

You have to organize the array in a way that all the even numbers come to the front and all odd numbers are pushed to the back. For example, the array B from above becomes Org_B : {6, 2, 4, 9, 11, 7}. Note that numbers don't need to be sorted here.

a. Propose a linear-time [O(N)] algorithm with constant [O(1)] space complexity. Present your algorithm as pseudocode or executable code, or step-by-step logical instructions. If unable, show a simulation of any algorithm (no limit on the time or space complexity) on the given example array B for 40% marks.

b. After solving 7(a), you want to find the index of the rightmost even number in the organized array Org_B. Present a modification of Binary Search to achieve this. Write either pseudocode or executable code.

Divide-and-Conquer Algorithms, Karatsuba Multiplication,
and Recursive Time-Complexity Analysis

---

1. Consider the following recursive function:

```
int recur(int A[], int p, int r)
{
    int n = r - p + 1;  // Size of the array

    if (n <= 1)
        return 2;

    int b = recur(A, p, p + (n / 4));
    int c = recur(A, p + (n / 4) + 1, p + (n / 2));
    int d = recur(A, p, p + (n / 3));

    int e = foo(n);  // Time complexity of foo(n): O(n log n)

    return b + c - d + e;
}
```

Here, p denotes the starting index and r denotes the ending index of array A[].

Write the recurrence relation for recur() and solve it using an appropriate method to determine its asymptotic time complexity.

2.

a. Consider the recurrence relation $T(n) = 4T(\sqrt[3]{n}) + O(n)$. Can we solve this using the Master Theorem? Explain whether the structure of this recurrence fits the requirements of the Master Theorem.

b. You are given the following recurrence relation:

$$T(n) = 3T\left(\frac{n}{4}\right) + \Theta(3n^2)$$

Solve the recurrence relation using an appropriate method

3. We are given an array A consisting of N integers. Some of the numbers are positive, and some are negative. For example $A: \{15, -13, 3, -4, 5, -6\}$.

We want to find the **Max-Prod Subarray** of the input. Just like the Max-sum Subarray, but instead of summation, we have to consider the product (multiplication) of the numbers.

We have come up with a divide-and-conquer algorithm to solve this problem. This algorithm has the same concept that you know for the Max-sum Subarray, just modified for multiplication.

```
function MaxProdSubarray(A, left, right){
    if (left == right):
        return A[left]

    mid = (left + right) / 2
```

```
    LPS = MaxProdSubarray(A, left, mid)
    RPS = MaxProdSubarray(A, mid + 1, right)
    CPS = CrossProduct(A, left, mid, right)

    return max(LPS, RPS, CPS)
}

function CrossProduct(A, left, mid, right){
    leftProd = -∞
    prod = 1
    for (i from mid to left){
        prod = prod * A[i]
        if (prod > leftProd): leftProd = prod
    }

    rightProd = -∞
    prod = 1
    for (i from mid+1 to right){
        prod = prod * A[i]
        if (prod > rightProd): rightProd = prod
    }
    return leftProd * rightProd
}
```

a. Mark the changes that you see in this algorithm than the one for Max-Sum Subarray. You can either write only those lines in your script. Or, write the whole code given above and underline the ones you want to mark. Also, you don't have to mark the changes in variable and function names.

b. Evaluate whether the given algorithm would actually work for this problem. If your answer is yes, give a proper logical explanation. Otherwise, give an input where the algorithm will not produce the correct answer.

c. We are thinking, why not use Karatsuba's multiplication technique to make our algorithm faster? So we replace the multiplication operations (*) with Karatsuba's function. For example,

prod*A[i] becomes Karatsuba(prod, A[i]).

Write a recurrence relation [T(N)= ____ + ____] to express the time complexity of the algorithm after applying Karatsuba. Assume that Karatsuba's time complexity is O(N1.58)(same N as the size of the Array A).

d. Solve the recurrence obtained in part (c) using an appropriate method and determine the asymptotic time complexity of the modified algorithm. Show the necessary steps.

Graph Theory - I

1. Consider the following everyday morning activities:

➔ Waking Up
➔ Brushing
➔ Leaving the house
➔ Taking a shower
➔ Getting dressed
➔ Having breakfast

Constraints:
- You must not get dressed before taking a shower
- You are free to have breakfast and then brush, or the opposite
- After waking up: you can either take a shower and then brush, or the opposite
- You must not leave the house without getting dressed
- Waking up is the first thing you do, and leaving the house is the last activity

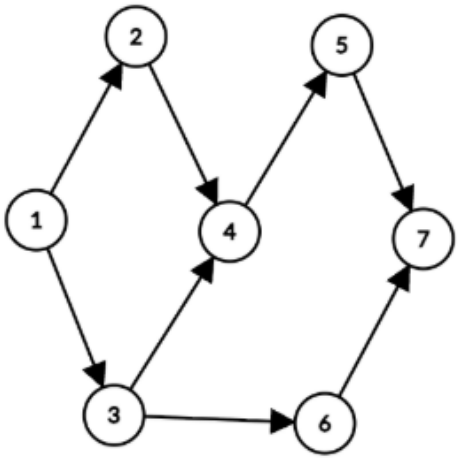
   A. Draw a directed graph representing the activities as nodes, and constraints as edges.
   B. From the above activities, write two different topologically sorted sequences of activities.
   C. Explain in short, how you would use DFS to topologically sort the activities.

2. The following information is stored after implementing the DFS algorithm on an undirected graph.

Nodes	1	2	3	4	5	6	7
Parent							
Starting Time	8	11	5	6	4	1	2
Finish Time	9	12	10	7	13	14	3
Distance from Root							

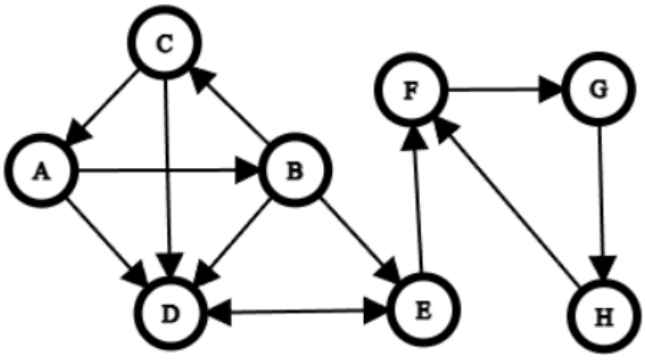
   A. Draw the DFS tree.
   B. Fill up the empty rows of the table.

3. A software build system manages modules {1, 2, 3, 4, 5, 6, 7} where a directed edge u→v means module u must be compiled before module v. The dependency graph G has been provided below:



A. Simulate DFS on G starting from module 1, visiting neighbors in numerical order (whenever there is a tie). Record discovery time d[v] and finish time f[v] for each module.
B. A valid compilation order requires every module to appear after all its dependencies. Using the finish times from Q(a), determine one such order.
C. A developer adds a new dependency 7→3, forming graph G'. Identify all cycles from G'. Just write your answer, no need to show any simulation.
D. Since G' contains cycles, a topological sort no longer exists. To fix the build order, merge all mutually dependent modules that form cycles into a single node. Modules not in any cycle remain as individual nodes. Using the cycles from Q(c), list the members of each merged group and draw the resulting dependency graph.

4. Consider the directed graph given below.



A. Find out the minimum number of edges to remove from the graph, so that there exists a topological order among the vertices. Explain your reasoning.

B. Using an appropriate algorithm, identify all strongly connected components (SCCs). Then, treat each SCC as a single vertex, draw the resulting graph, and determine whether any cycles remain

5. A. For a directed graph, does the number of back edges equal the number of cycles present? Explain briefly
   B. Given a directed graph G = (V, E) represented with an adjacency list, explain how you can find whether the graph is a directed acyclic graph (DAG) or not. Your method must be as asymptotically efficient as possible.
   C. Given a set of SCCs, what is the maximum number of new SCCs that can appear if we delete one edge from the DAG of these SCCs?

Assignment - 4

1. a. During the July festival, organizers want to arrange a drone show. There will be N drones carrying lights over an N × N aerial stage. The drones must form a symmetrical light display, but no two drones can align vertically or horizontally.

   If the number of drones is 6, draw a valid arrangement of drones in a 6x6 grid.

   b. A delivery robot works in a storage area organized in rows and columns of delivery packages, where each package can be thought of as a cell. There is only one free spot (cell) that allows the robot to slide packages around. Its goal is to arrange all the packages in a specific order so that they are ready for shipping.

   While trying different moves, the robot keeps going back to layouts it has already tried, getting stuck in a loop and making no progress.

   1. How can the robot avoid repeating the same arrangements again and again? Explain within two sentences.

   2. Name an algorithm that will help to find the shortest sequence of moves to reach the correct order of packages.

2. A sensor network monitors environmental conditions and transmits different types of sensor readings. Each reading has a frequency of occurrence and an impact factor representing its importance in critical decision-making. The system uses variable-length encoding to compress the readings and aims to maximize the total impact factor transmitted within its data capacity.

   Sensor Data Table

Sensor Reading	**Frequency of Occurrence**	**Impact Factor**
Temperature	45	10
Humidity	13	6
Pressure	12	8
Light	16	7
CO_2	9	5
Wind Speed	5	4

Rainfall	7	6
UV Index	3	3

a. Construct a Huffman tree for the sensor readings based on their frequencies of occurrence. Assign a binary code to each reading.

b. Calculate the total number of bits required to transmit all the sensor readings using the Huffman codes obtained in part (a).

c. Suppose the system can transmit only 15 units of data. Transmitting all occurrences of a sensor reading requires data units equal to its frequency of occurrence. Partial transmission is allowed, with the obtained impact factor decreasing proportionally.
Determine which sensor readings should be transmitted, either fully or partially, to maximize the total impact factor within the 15-unit capacity. Specify the amount of each selected reading to be transmitted and calculate the maximum total impact factor.

3. You are given two strings, 'cbabdcb' and 'badcbc', and tasked with finding a common sequence with the longest length. Initially, you considered a recursive solution, but person 'X' suggested using a dynamic programming approach instead.

a. Find the longest common sequence using the approach proposed by person ‘X’. Show the necessary simulation to find the answer.

b. Additionally, you want to send these strings as an encoded message. After merging them into one string, 'cbabdcbbadcbc', now you need an efficient text-encoding algorithm with minimal space requirement.

i. Simulate the encoding process,

ii. construct any needed tables or trees,

iii. generate the codeword for each character,

iv. show the encoded message and

v. the total number of bits required to store it.

4. You are attending the CSE221 final examination, where each question requires a different amount of time and carries a different number of marks. After answering several questions, you realize that only 6 minutes remain. Your goal is to maximize the total marks obtained without exceeding the remaining time.

Question Information

- **Question 4: Time = 5 minutes, Marks = 8**
- **Question 5: Time = 3 minutes, Marks = 7**
- **Question 6: Time = 4 minutes, Marks = 12**
- **Question 7: Time = 6 minutes, Marks = 10**
- **Question 8: Time = 2 minutes, Marks = 6**
- **Question 9: Time = 1 minute, Marks = 5**

a. [CO3] Suppose no partial marks are awarded for incomplete answers. Determine the combination of questions that maximizes your total marks without exceeding the 6-minute time limit. Use an efficient approach instead of brute force, and show the complete calculation.

b. [CO3] The faculty announces that partial marks will now be awarded for partially completed answers. The marks received will be proportional to the amount of time spent on a question. For example, completing half of a four-minute question will award half of its marks.

Determine the order in which you should attempt the questions to maximize your score within the 6-minute limit. Calculate the maximum marks obtainable and compare the result with part (a).

c. [CO2] Explain briefly why dynamic programming can have a lower time complexity than brute-force or basic recursive solutions.